

S²-MoE: Enabling Efficient Self-Speculative Decoding for Mixture-of-Experts on Edge Devices

Haochen Huang^{1,2}
hhc@stu.pku.edu.cn

Shengxuan Qiu^{1,2,3}
2300012877@stu.pku.edu.cn

Meng Li^{1,2*}
meng.li@pku.edu.cn

Abstract

Deploying large language models (LLMs) for inference on edge devices is challenging due to severe memory and bandwidth constraints. While speculative decoding and Mixture-of-Experts (MoE) have been proposed to improve inference efficiency, naively combining them often incurs excessive verification overhead and poor expert reuse, limiting their effectiveness in memory-bound edge settings. In this work, we propose S²-MoE, an efficient self-speculative decoding framework for MoE inference on edge devices. S²-MoE reduces redundant verification through routing-aware adaptive speculative expansion, improves verification efficiency with reuse-aware expert gating, and aligns draft and target execution via shared context. Implemented in llama.cpp, S²-MoE achieves up to 5.3× speedup (about 2.0× on average) over standard autoregressive decoding across diverse MoE models and datasets on edge devices. Code is available at <https://github.com/angerybob/S2-MoE>.

1 Introduction

Large language models (LLMs) have achieved remarkable performance across a wide range of tasks. However, many privacy-sensitive and latency-critical applications require LLM inference to be performed directly on edge devices. In such edge settings, the rapidly growing scale of LLMs poses significant challenges due to the limited resources available on edge platforms [10, 12, 17, 27, 41].

A key bottleneck in edge LLM inference is memory bandwidth [22]. Edge inference workloads typically operate with small batch sizes and often degenerate to single-batch execution, resulting in limited parameter reuse and severe memory bandwidth pressure.

Under this setting, throughput is largely determined by how many tokens can be generated per access to model parameters. This efficiency is influenced by two intuitive factors: the number of tokens generated per iteration, referred to as **output density**, and the amount of model parameters accessed per iteration, referred to as **parameter density**, leading to the following decomposition:

$$\text{Throughput} = \frac{\text{Output Density (tokens/iter.)}}{\text{Parameter Density (GB/iter.)}} \cdot \text{BW} \quad (1)$$

where the bandwidth BW is limited in resource-constrained devices.

Building on this decomposition, existing edge inference acceleration techniques can be broadly categorized according to how they optimize these two factors. On one hand, **speculative decoding** (SD) [15, 20, 23, 32, 50] accelerates autoregressive inference by first generating draft tokens with a lightweight draft model and then

verifying them in parallel with the full model (often referred to as the target model). By allowing multiple tokens to be produced within a single iteration, speculative decoding effectively increases the output density. On the other hand, **model sparsification** [51], such as **Mixture-of-Experts** (MoE), reduces inference cost by activating only a subset of model parameters per iteration. This selective execution directly reduces the parameter density.

Despite their complementary benefits, SD and MoE are difficult to combine effectively in edge inference settings. As illustrated in Figure 1, speculative decoding relies on high parameter reuse during verification. While this property naturally holds for dense models, it breaks down for MoE models, where draft tokens may activate diverse experts, resulting in significantly increased verification cost. This weakens the advantage of speculative decoding in memory-bound regimes. More critically, MoE models are highly sensitive to verification cost, since draft tokens that are ultimately rejected may still trigger disjoint expert activations, making verification disproportionately expensive. As a result, naively applying speculative decoding to MoE models can negate, or even reverse, the efficiency gains of either technique when used in isolation.

Most state-of-the-art SD methods improve throughput by generating and jointly verifying a large set of draft tokens to achieve sufficient acceptance length. Representative approaches such as EAGLE-3 [23] rely on confidence-guided expansion and pruning to control speculative width. While such strategies can suppress some low-quality branches, they remain difficult to calibrate in MoE settings. Token-level confidence does not reflect expert-level verification cost, and even draft candidates with similar confidence may incur substantially different verification overhead since they may trigger disjoint expert activations. Moreover, existing draft models are typically designed to be lightweight to keep drafting overhead low, but this often limits predictive fidelity, making it difficult to simultaneously achieve long acceptance lengths and low verification cost.

In this context, **self-speculative decoding** [2, 11, 26, 46–48] offers a unique opportunity to better align draft and target behaviors for speculative decoding in MoE models. Instead of relying on an external draft source, prior self-speculative approaches construct a lightweight draft directly from the target model itself. Because the draft is derived from the same underlying model, its prediction behavior is naturally better aligned with that of the target. This stronger alignment makes self-speculative decoding a more promising starting point for achieving long acceptance lengths with a lightweight draft, while also reducing redundant verification.

While self-speculative decoding offers strong potential for aligning draft and target behaviors, it is not directly applicable to MoE

*Corresponding author.

¹Institute for Artificial Intelligence, Peking University, Beijing, China

²School of Integrated Circuits, Peking University, Beijing, China

³School of Electronics Engineering and Computer Science, Peking University, Beijing, China

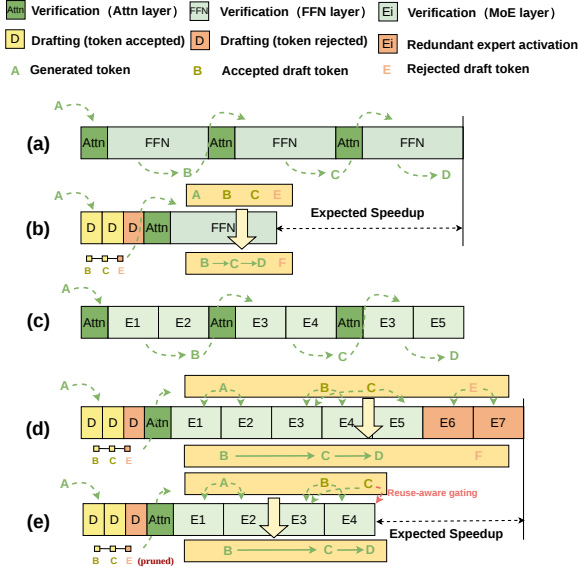


Figure 1: Comparison of autoregressive and speculative decoding on dense and MoE models. (a) Autoregressive decoding on a dense model. (b) SD on a dense model, where draft tokens can be efficiently verified due to parameter reuse across tokens. (c) Autoregressive decoding on a MoE model. (d) Naïve SD on a MoE model, where draft tokens activate different experts, leading to low parameter reuse and high verification cost. (e) S^2 -MoE improves SD on MoE models by reducing redundant verification and improving parameter reuse.

models, as achieving sufficient lightweight drafts inevitably introduces fidelity loss. First, imperfect alignment between draft and target still provides limited acceptance rates. Second, rejected drafts lead to excessive redundant verification, eroding the benefits of speculative decoding. Third, MoE models inherently exhibit limited expert parameter reuse, and diverse expert activations across speculative tokens further exacerbate this issue, significantly increasing verification cost. Therefore, a design that simultaneously ensures high draft fidelity, low redundant verification, and strong parameter reuse is required for SD in MoE models.

To meet this requirement, we propose S^2 -MoE, an efficient self-speculative decoding framework for MoE inference on edge devices (Figure 1(e)). In summary, we contribute: (1) **Routing-aware Adaptive Speculative Expansion**, which dynamically expands speculative candidates based on their *verification utility*, allowing the system to avoid low-quality candidates with high verification cost and thereby reducing redundant verification; (2) **Reuse-aware expert gating**, a MoE gating design that explicitly promotes expert parameter reuse across draft tokens, thereby increasing the computational density of verification; (3) **Context-aligned self-speculative decoding**, which enables the lightweight draft and the target model to *share the same decoding context KV Cache*, preventing approximation errors in the draft from accumulating across iterations and thereby improving acceptance rates; (4) A full system implementation integrated into `llama.cpp`, demonstrating end-to-end efficiency gains. Across diverse MoE models and datasets,

S^2 -MoE achieves 1.3× to 5.3× speedup on Jetson Orin and 1.2× to 2.9× on RTX 4090 over standard autoregressive decoding. Our code is available at <https://github.com/angerybob/S2-MoE>.

2 Background

2.1 Mixture-of-Experts

Mixture-of-Experts (MoE) models improve scalability by sparsely activating a small subset of experts per token, reducing effective computation and parameter access compared to dense models. This sparse execution enables large-capacity models to operate under lower average memory and compute cost, making MoE architectures attractive for memory-constrained inference scenarios, including edge LLM deployment [1, 24, 25, 42–44, 51].

2.2 Speculative Decoding

Speculative decoding (SD) [15, 20, 23, 32, 50] accelerates autoregressive decoding by using a lightweight mechanism to propose future tokens and letting the target model verify them in parallel, which is particularly beneficial in memory-bound settings. Existing SD methods mainly differ in how speculative candidates are generated, including methods based on an external draft model, methods using internal features or auxiliary prediction heads, and parallel decoding methods without an explicit draft.

For MoE models, however, these approaches remain difficult to apply effectively. External-draft methods require a draft model that is both lightweight and well aligned with the target [20, 32], which is difficult to achieve in practice. This challenge is even more pronounced for MoE models, where only a small subset of parameters is already activated per token, leaving limited room to construct a substantially smaller yet well-aligned draft. Methods based on auxiliary heads or parallel decoding [15, 23, 50] avoid an external draft, but typically still require generating and verifying many speculative candidates to maintain a useful acceptance length. In MoE models, this can be especially expensive because different candidates may activate different experts, leading to high and redundant verification overhead. Although confidence-based expansion and pruning (e.g., EAGLE-3 [23]) can reduce some low-quality branches, token-level confidence does not directly reflect expert-level verification cost. Moreover, the effectiveness of such predictors often depends heavily on the model family and predictor training quality, which can make it difficult to consistently sustain long acceptance lengths and large speedups across MoE models.

2.3 Self-Speculative Decoding

Motivated by this gap, self-speculative decoding eliminates the need for an external draft by deriving a lightweight draft directly from the target model itself, thereby achieving strong behavioral alignment. Existing self-speculative approaches typically construct such drafts through various forms of model simplification. Common strategies include quantization [48], expert sparsity [2], which activates fewer experts per token in MoE models, and layer sparsity [11, 26, 47], which skips selected layers or enables early exit during decoding. These techniques form a spectrum of trade-offs: while more aggressive simplification reduces draft execution cost, it also degrades prediction quality and limits acceptance rates.

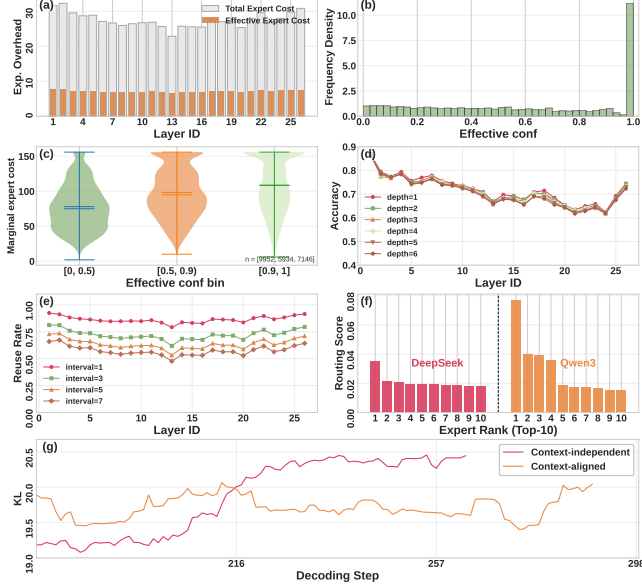


Figure 2: (a) Layer-wise breakdown of verification cost for DeepSeekMoE evaluated in Natural Questions, highlighting redundant expert activation. (b) Distribution of Draft Confidence. (c) Verification Cost Distribution Across Confidence Levels. (d) Accuracy of draft expert routing in predicting target routing under different draft depths. (e) Expert reuse ratio across layers under different decoding intervals. (f) Distribution of top routing scores, revealing expert redundancy. (g) KL divergence between draft and target models over decoding steps w/ and w/o shared context.

2.4 Speculative Decoding for MoE Models

Existing studies have explored applying SD to MoE models from multiple perspectives. Some works leverage SD to predict future expert activations, enabling expert prefetching and offloading, but primarily focus on system-level caching and scheduling without addressing the intrinsic inefficiencies of verification under sparse activation [5, 6, 21, 37, 38, 49, 52]. Other efforts observe that SD is not universally beneficial for MoE inference and propose utility- or cost-aware policies to dynamically enable speculation and adjust the speculation length. However, such approaches are driven by historical utility signals, whose reliability can degrade under unstable routing, limiting their ability to consistently improve speculative efficiency [35]. More recently, SD has been shown to provide speedups for MoE models under moderate batch sizes by amortizing verification overhead across requests, yet these benefits largely diminish in edge inference settings, where batch sizes are typically one or two and verification remains memory-bound [18]. Importantly, these studies primarily provide empirical and analytical insights into this behavior, rather than addressing the underlying inefficiencies.

3 Motivation

Challenge 1: Excessive redundant verification under low-fidelity drafts. In MoE models, verifying each draft token may

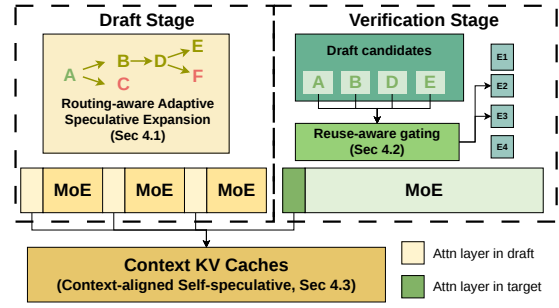


Figure 3: S^2 -MoE overview.

activate additional experts, incurring nontrivial parameter access. When draft tokens are rejected, these extra activations become redundant verification cost. Fig. 2(a) shows that, across layers, a large fraction of verification cost is spent on experts triggered by ultimately rejected tokens, highlighting the inefficiency of naïve speculation. Moreover, this redundancy is difficult to eliminate with confidence-only pruning. As shown in Fig. 2(b), draft confidence is highly concentrated near 1.0 for retained tokens, making confidence weakly discriminative in practice. At the same time, Fig. 2(c) shows that these tokens can still incur substantially different verification costs in MoE models due to different expert activation patterns. This motivates speculative expansion that jointly considers acceptance potential and verification overhead.

Opportunity 1: Predicting verification cost via routing similarity. As illustrated in Fig. 2(d), draft-time expert routing predicts target routing with high accuracy in many layers, enabling draft routing signals to serve as a proxy for estimating verification cost. This motivates adaptive speculative expansion that explicitly balances potential acceptance benefit (typically estimated from the draft’s token-level confidence as in prior work) against expected verification overhead.

Challenge 2: Limited expert parameter reuse under diverse expert activations. Unlike dense models, where adjacent tokens share the same parameters, MoE models often activate different experts for neighboring tokens. We measure expert reuse using a *reuse ratio*, defined for a token interval of length s as the number of unique experts activated within that interval during parallel verification, normalized by the number of expert activations in autoregressive decoding over the same interval. A smaller reuse ratio indicates stronger cross-token expert reuse. In the ideal case where all tokens within the interval reuse the same set of experts, the ratio approaches $1/s$, which corresponds to a dense-like parameter reuse pattern across tokens. As shown in Fig. 2(e), this ratio remains high across layers under different decoding intervals, indicating limited expert reuse and substantially higher parameter traffic under parallel verification.

Opportunity 2: Flexible routing among low-impact experts. In MoE models, expert importance is often skewed, with a few high-impact experts receiving dominant gating scores, while others receive lower scores. Experts with similar low scores, though not selected, can often perform similarly. Fig. 2(f) shows that beyond the top-ranked experts, many candidates exhibit comparable routing scores, indicating functional redundancy. This creates an opportunity to trade a small loss in per-token gating score for increased

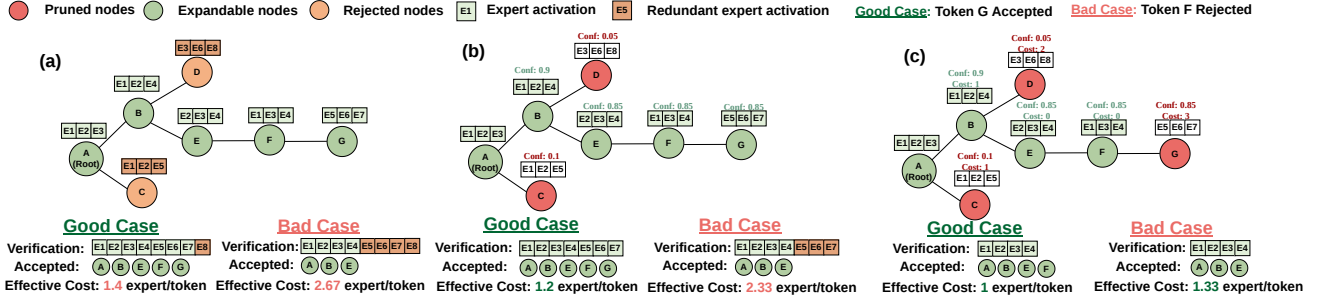


Figure 4: Routing-aware adaptive expansion reduces verification cost under different acceptance outcomes. (a) Expansion without pruning. (b) Confidence-based pruning. (c) Our routing-aware adaptive expansion.

expert reuse, by allowing a token to select experts with slightly lower scores that are already activated by other draft tokens.

Challenge 3: Limited draft fidelity under lightweight draft. Although self-speculative drafts are derived from the target model itself, lightweighting still introduces prediction errors. When the draft maintains an independent context, these errors accumulate over decoding steps, causing progressive divergence and reducing acceptance rates in long-form generation. As shown in Fig. 2(g), the KL divergence between draft and target distributions grows rapidly in this case when the draft maintains an independent context, indicating severe error accumulation over time.

Opportunity 3: Reducing error accumulation via shared context. In autoregressive language models, next-token prediction follows $P(y_t | x, y_{<t})$, so even small errors in the conditioning context can affect subsequent predictions and accumulate over decoding steps. By sharing the same decoding context between the draft and target, such errors are confined to the current speculative step instead of propagating across iterations. As shown in Fig. 2(g), context alignment substantially suppresses divergence growth over long contexts.

4 S²-MoE Design

Figure 3 illustrates the overall design of S²-MoE. Our framework combines three complementary components to improve speculative decoding for MoE models on edge devices: routing-aware adaptive speculative expansion (Sec. 4.1), reuse-aware expert gating (Sec. 4.2), and context-aligned self-speculative decoding (Sec. 4.3). We next describe these components in turn.

4.1 Routing-aware Adaptive Speculative Expansion

To reduce redundant verification in MoE speculative decoding, we adaptively expand the draft tree according to a utility score that jointly accounts for acceptance likelihood and verification cost. This design is motivated by a key limitation of prior adaptive expansion rules: in dense models, token confidence is often a reasonable proxy for speculative utility, but in MoE models, the verification cost depends heavily on which experts are activated. As a result, two draft candidates with similar confidence can induce substantially different verification overhead if one largely reuses already activated experts while the other introduces many new ones.

Expected Benefit. Following prior speculative decoding works, for a draft token i at depth k , we estimate its potential acceptance

probability as the prefix confidence $p_i = \prod_{j=1}^k p_{i,j}$, since token i can be accepted only if all preceding draft tokens are also accepted. If accepted, token i saves one autoregressive decoding step of the target model. We therefore define its expected benefit as $B_i = p_i \cdot T_{AR}$, where T_{AR} denotes the average latency of one target-model decoding step.

Routing-aware Cost Estimation. The cost of expanding token i consists of two parts: (i) the draft expansion cost for generating its descendants, and (ii) the verification cost incurred when validating it with the target MoE model. Unlike dense models, where verification cost increases little with the number of draft tokens, MoE verification cost increases significantly as more tokens are validated, since each additional token may activate a new set of experts. Let $\mathcal{E}_i$ denote the predicted experts for token i , and let S denote the set of draft tokens already selected for expansion. We define the marginal verification cost term of adding token i as

$$\widehat{\Delta C}^{\text{ver}}(i | S) = \left| \mathcal{E}_i \setminus \bigcup_{j \in S} \mathcal{E}_j \right| \cdot T_{\text{exp}}, \quad (2)$$

where T_{exp} denotes the latency associated with introducing one additional expert during verification and can be obtained through lightweight runtime measurement or offline profiling. This term converts newly introduced experts into a hardware-calibrated latency unit, enabling direct comparison between verification cost and expected benefit. This estimate uses draft-time routing signals, which are available before verification and correlate with target routing across layers, as demonstrated in Opportunity 1. As shown in Fig. 6, the resulting cost model closely tracks measured verification latency across models and memory budgets ($R^2=0.943-0.999$, MAPE=0.8–4.5%), indicating that it reliably predicts the actual verification overhead used by adaptive expansion. The total expansion cost is then defined as $C_i = \widehat{\Delta C}^{\text{ver}}(i | S) + C_i^{\text{draft}}$.

Adaptive Expansion. The utility of a draft token i is defined as

$$U_i = \frac{B_i}{C_i} = \frac{p_i \cdot T_{AR}}{\widehat{\Delta C}^{\text{ver}}(i | S) + C_i^{\text{draft}}}. \quad (3)$$

Speculative expansion proceeds greedily by admitting candidates with $U_i \geq 1$, i.e., when the expected benefit exceeds the estimated cost, until no further candidates satisfy the criterion.

Compared with fixed-width or confidence-only expansion, our routing-aware utility captures a MoE-specific effect that prior methods overlook: under MoE verification, candidate quality depends

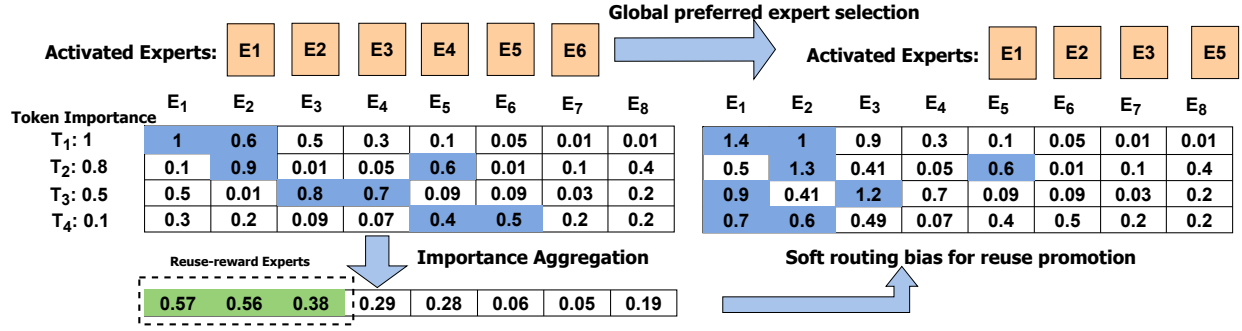


Figure 5: An illustrative example of reuse-aware expert gating with top-2 routing. Four draft tokens select two experts each from eight experts. Token-level confidence is used to aggregate expert importance and identify a small set of reuse-reward experts (E_1 – E_3), whose logits are softly biased to improve cross-token expert reuse during verification.

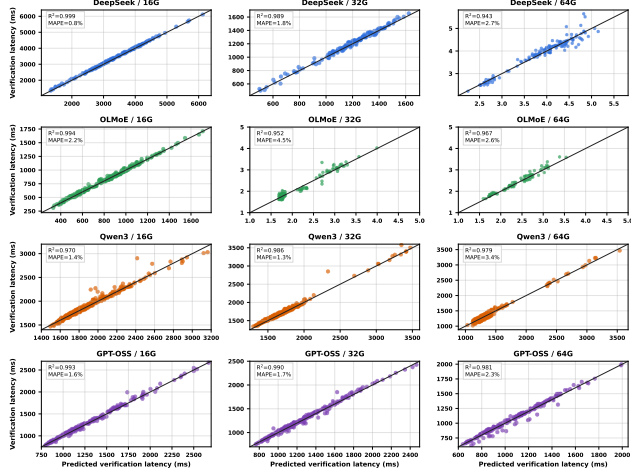


Figure 6: Cost-model calibration against measured verification latency across models and memory budgets.

not only on acceptance likelihood but also on marginal expert activation cost. As illustrated in Fig. 4, expansion without pruning (a) verifies all draft candidates, leading to substantial redundant expert activation, while confidence-based pruning (b) removes clearly low-quality branches but remains blind to expert-level verification cost and may still retain high-cost candidates in some cases. In contrast, our routing-aware adaptive expansion (c) jointly considers acceptance likelihood and marginal verification cost, favoring candidates with better acceptance-to-verification trade-offs and reducing redundant expert activation. This advantage holds even when acceptance outcomes are unfavorable (Bad Case in Fig. 4), as prioritizing low-cost candidates still limits redundant overhead.

4.2 Reuse-aware Expert Gating

In contrast to dense models, where neighboring tokens reuse parameters, MoE models often route consecutive tokens to different experts. This effect is amplified under speculative decoding with parallel verification, resulting in heavy expert activation and reduced parameter reuse, which increases memory overhead.

To address this problem, we introduce **reuse-aware expert gating**, a gating mechanism that softly aligns expert routing decisions across speculative tokens. The core idea is to first identify experts that already exhibit repeated activation across a verification batch, and then softly amplify this existing reuse tendency, thereby improving expert parameter locality and reducing memory traffic.

Cross-token expert importance aggregation. Tokens differ in their likelihood of being accepted during verification. High-confidence tokens are more likely to be accepted and therefore should preserve their original expert preferences, whereas low-confidence tokens are less likely to survive during verification and mainly contribute to redundant computation. To reflect this asymmetry, we associate each token with a non-negative confidence weight w_b , derived from its token-level confidence, so that expert preferences of high-confidence tokens receive greater emphasis in aggregation.

Using these confidence weights, we consider a set of draft tokens $\mathcal{B} = \{1, \dots, B\}$ and an MoE layer with expert set $\mathcal{E} = \{1, \dots, E\}$. For each token $b \in \mathcal{B}$, let $\ell_b \in \mathbb{R}^E$ denote the expert gating logits produced by the target model. We aggregate expert importance across speculative tokens as

$$g_e = \sum_{b \in \mathcal{B}} w_b \cdot \ell_{b,e}, \quad \forall e \in \mathcal{E}, \quad (4)$$

where g_e captures how strongly expert e is favored by speculative tokens that are more likely to be accepted. This aggregation jointly accounts for token-level confidence and expert preference, thereby identifying experts that are most valuable to prioritize for reuse during verification.

Global preferred expert selection. To avoid over-concentration and preserve the original top- k routing structure, We restrict biasing to a bounded set of experts. Specifically, we select a set of *globally preferred experts* $\mathcal{E}^* = \text{TopCap}(\{g_e\}_{e=1}^E)$, where $|\mathcal{E}^*| \leq \text{cap}$ and $\text{cap} \geq k$. This ensures that all originally eligible top- k experts remain selectable, while limiting the scope of routing bias.

Soft routing bias for reuse promotion. For every speculative token b , we inject an additive bias into its gating logits $\ell'_{b,e} = \ell_{b,e} + \lambda_b \cdot 1[e \in \mathcal{E}^*]$, where λ_b is determined based on the routing score distribution of token b . Specifically, we set λ_b proportional to the gap between the top-1 expert and the $(k+1)$ -th expert. This gap provides a natural measure of the margin for entering the top- k set, allowing

the bias to promote reuse-reward experts into the selection boundary without overly perturbing dominant routing decisions. The required routing scores are readily available from the gating module during inference, and can also be obtained through lightweight offline profiling, introducing no additional training requirement. In practice, the bias primarily affects borderline experts near the top- k threshold, while preserving the relative ordering of strongly preferred experts, thereby improving expert reuse without significantly degrading routing quality. Moreover, because this bias increases expert overlap across draft tokens, the realized verification cost is generally no higher than that estimated from the original draft routing, making the cost estimate in Sec. 4.1 conservative.

Efficient fused implementation. We implement reuse-aware expert gating as a lightweight fused CUDA kernel, introducing negligible overhead during speculative decoding.

Figure 5 illustrates reuse-aware expert gating with a toy example. Routing information from four draft tokens with different confidence levels is aggregated to identify a small set of reuse-reward experts (e.g., E_1 – E_3), whose gating logits are softly biased. This bias reduces the number of activated experts during verification (from six to four in this example) while remaining token-adaptive: high-confidence tokens (e.g., T_1 and T_2) largely retain their original expert choices, whereas low-confidence tokens (e.g., T_4) are more likely to shift toward reuse-reward experts.

4.3 Context-aligned Self-speculative

A major source of fidelity loss in self-speculative decoding is context misalignment between the draft and target models, where independently maintained decoding states cause errors to accumulate over time. To address this issue, we propose context-aligned self-speculative decoding, in which the draft and target share a unified KV cache.

Specifically, the target model first processes the prompt and establishes the KV cache. During speculative generation, the draft directly reads from this shared context instead of maintaining a separate history, eliminating context drift. To preserve correctness, KV updates from the draft are treated as temporary and rolled back before verification, and only accepted tokens are committed to the shared cache.

This design ensures that draft errors are confined to individual speculative steps and do not accumulate across iterations.

5 System Implementation

Framework. We implement our approach on top of **llama.cpp**, a widely used inference framework for edge and resource-constrained deployment. All proposed mechanisms in S^2 -MoE are fully integrated into the existing decoding pipeline.

Edge memory constraints and expert-level offloading. Edge platforms often operate under tight memory budgets, especially when serving large MoE models. We implement expert-level offloading on both NVIDIA Jetson Orin and a discrete-GPU NVIDIA RTX 4090 platform. On Jetson Orin, the CPU and GPU share a unified DRAM without dedicated host memory, so parameters exceeding device memory are offloaded to lower-tier storage such

as SSDs. On RTX 4090, with separate CPU/GPU memories, cold experts are offloaded to CPU memory and transferred on demand.

While **llama.cpp** natively supports layer-level parameter offloading, this granularity is ill-suited for MoE models because every layer is visited during inference, but only a small subset of experts is activated per token. Offloading entire layers therefore incurs unnecessary data movement. To address this mismatch, we implement expert-level parameter offloading, where non-expert and shared parameters remain resident in device memory, and individual expert parameters are dynamically fetched on demand. When memory permits, we further keep frequently activated experts resident in device memory based on offline profiling, while ensuring sufficient runtime memory headroom for online inference.

Our work focuses on improving speculative decoding efficiency and is orthogonal to prior efforts on parameter offloading and caching strategies. Existing offloading methods primarily optimize data movement through prefetching and cache management, and their ideal operating regime corresponds to scenarios with high cache hit rates. Such settings are effectively equivalent to memory-relaxed configurations, which we explicitly evaluate (e.g., 64GB memory budgets).

Moreover, to the best of our knowledge, no existing MoE offloading work provides a publicly available implementation compatible with **llama.cpp**. Re-implementing these systems would require substantial engineering effort beyond the scope of this work.

Baseline models and implementations. We evaluate S^2 -MoE against state-of-the-art SD baselines implemented on representative MoE model families. For GPT-OSS, Qwen3, and OLMoE models, we use publicly released EAGLE-3 checkpoints from Hugging Face, including lmsys/EAGLE3-gpt-oss-120b-bf16 [28], AngelSlim/Qwen3-a3B_eagle3 [3], and wantsleep/OLMoE_1B_7B_Eagle3 [39]. For DeepSeek models, EAGLE-3 style speculative decoding is implemented based on the open-source SpecForge framework built on sglang [36]. All EAGLE-3 baselines are executed using the official EAGLE-3 implementation in **llama.cpp**, based on the corresponding development branch (see PR #18039) [14], which we adapt and integrate into our evaluation pipeline. For Cascade [35], we implement its core policy in **llama.cpp** following the method described in their paper, and integrate it into the same evaluation pipeline for a fair comparison.

Draft configuration. Our draft is constructed based on expert sparsity because quantization often incurs substantial draft overhead, while aggressive layer sparsity can severely degrade draft fidelity. The detailed configurations are summarized in Table 1. When memory permits, we additionally apply quantization to the sparse draft to further reduce computation and memory footprint. The draft length is fixed at 8, and the draft width is set to 2 for baseline models, while S^2 -MoE uses adaptive expansion based on utility scores.

Table 1: Model and SD configurations.

Model	Model configuration				SD hyperparameter	
	Total Params	Activated Params	Total Experts	Experts/Token	Cap	Draft Experts
OLMoE	6.7	1.1	64	8	18	2
DeepSeek	15.3	1.35	64	6	16	1
Qwen3	30.5	3.3	128	8	14	3
GPT-OSS	116.8	5.7	128	4	6	1

6 Experiments

6.1 Experimental Setup

Models. We evaluate our method on four representative MoE-based LLMs covering diverse model scales and design generations: DeepSeek-V2-Lite-Chat (DeepSeek) [24], OLMoE-1B-7B (OLMoE) [33], Qwen3-30B-A3B (Qwen3) [42], and GPT-OSS-120B (GPT-OSS) [1]. These models span from lightweight edge-friendly MoE models to large-scale, high-capacity MoE systems, enabling a comprehensive evaluation across different parameter sizes, expert configurations, and routing behaviors (see Table 1).

Datasets and Tasks. Following EAGLE-3 and Spec-Bench [40], we evaluate effectiveness across a diverse set of generation tasks, including multi-turn conversation (**MT**), retrieval-augmented generation (**RG**), summarization (**SU**), translation (**TR**), question answering (**QA**), mathematical reasoning (**MA**), and code generation (HumanEval, **HE**), using identical speculative decoding hyperparameters across all tasks for fairness. For quality evaluation, we assess whether reuse-aware gating introduces degradation in model quality. Specifically, DeepSeek and OLMoE are evaluated on GSM8K [9], HumanEval [7], ARC-E, and ARC-C [8], while the stronger Qwen3 and GPT-OSS are evaluated on GPQA-Diamond [34], AIME24 [29], AIME25 [30], and HMMT [16]. This split ensures that each model is evaluated on tasks aligned with its reasoning capability. We additionally report long-context quality on GovReport, NarrativeQA, and Qasper from LongBench [4], together with distribution-consistency metrics against the unmodified target on WikiText-2 [31].

Hardware Platform. We evaluate on two platforms: NVIDIA Jetson Orin edge devices and a discrete-GPU NVIDIA RTX 4090. On Jetson Orin, we consider three memory constraints: 16 GB, 32 GB, and 64 GB, corresponding to Jetson Orin NX 16GB and Jetson AGX Orin 32GB/64GB. When expert parameters exceed on-device memory capacity, we offload them to SSD storage, following common edge deployment practice. On RTX 4090, we evaluate under constrained GPU memory, where cold experts are offloaded to CPU memory.

Baselines. We compare against four baselines: (i) standard autoregressive decoding (**AR**); (ii) representative self-speculative decoding with expert sparsity (**ES**), which constructs a lightweight draft by activating fewer experts per layer than the target model; (iii) self-speculative decoding with layer sparsity (**LS**), which forms the draft by skipping a subset of transformer layers selected via Bayesian optimization. Due to its consistently poor efficiency in experiments, LS is evaluated only on DeepSeek and OLMoE, and omitted on Qwen3 and GPT-OSS where it is unlikely to be competitive; (iv) the state-of-the-art speculative decoding method EAGLE-3

(EAGLE3); (v) a representative MoE-aware speculative decoding approach (**Cascade**), which selectively enables speculative decoding during inference to balance draft benefit and verification overhead. For a stronger and more competitive baseline, we instantiate Cascade on top of EAGLE3 as the underlying speculative decoding engine.

6.2 Main Results

6.2.1 Efficiency. Fig. 7 summarizes the effectiveness of S^2 -MoE across different MoE models, tasks, and memory constraints. Since our goal is to improve end-to-end inference efficiency under memory-constrained edge deployment, we use **Speedup Ratio** relative to standard autoregressive decoding as the metric. Overall, S^2 -MoE consistently outperforms autoregressive decoding and other SD baselines, achieving 1.3–5.3 $\times$ speedup on Jetson Orin and 1.2–2.9 $\times$ on RTX 4090. Corresponding raw Tok/s, acceptance lengths, and speedups are reported in Appendix A.

Naïve self-speculative baselines often fail to outperform autoregressive decoding, particularly under tight memory budgets. Although self-speculative decoding improves alignment between draft and target, its benefits are largely offset by low expert reuse and excessive redundant verification, leaving substantial optimization headroom. In contrast, S^2 -MoE explicitly targets these bottlenecks, translating aligned drafts into effective speedups through reduced verification cost and improved parameter reuse.

Compared to EAGLE-3, S^2 -MoE achieves more stable gains across a diverse set of modern MoE models. While EAGLE-3 performs well on some model families (e.g., the Llama series), its effectiveness varies significantly with draft prediction quality and training characteristics. Despite being training-free, S^2 -MoE achieves consistently higher speedups across models and tasks.

Compared to Cascade, which decides whether to enable speculative decoding based on historical information, S^2 -MoE performs token-level speculative control using routing-aware cost estimation at the current step. This avoids the extra online decision overhead of test-and-set policies and provides a more direct estimate of verification utility, leading to more consistent speedup gains across models and tasks.

These advantages become more pronounced under tighter memory budgets, where more experts must be offloaded, and verification becomes increasingly sensitive to redundant expert activation and parameter reuse.

6.2.2 Quality. Reuse-aware expert gating intentionally introduces a bounded routing bias to improve expert reuse, and is therefore a controlled target-routing approximation rather than a strictly lossless transformation. Table 2 evaluates its quality impact against the unmodified Original model on both task accuracy and long-context / distribution-consistency metrics. On standard benchmarks, DeepSeek and OLMoE are evaluated on GSM8K, HumanEval, ARC-E, and ARC-C, while Qwen3 and GPT-OSS are evaluated on AIME24, AIME25, HMMT, and GPQA. We further evaluate long-context quality on three LongBench [4] datasets—GovReport, NarrativeQA, and Qasper—and measure output-distribution consistency against the unmodified target on WikiText-2 [31] via Mean KL, RMS logit shift (Δp), Top-1 agreement, and PPL ratio. Across models, task-accuracy differences remain small and non-systematic,

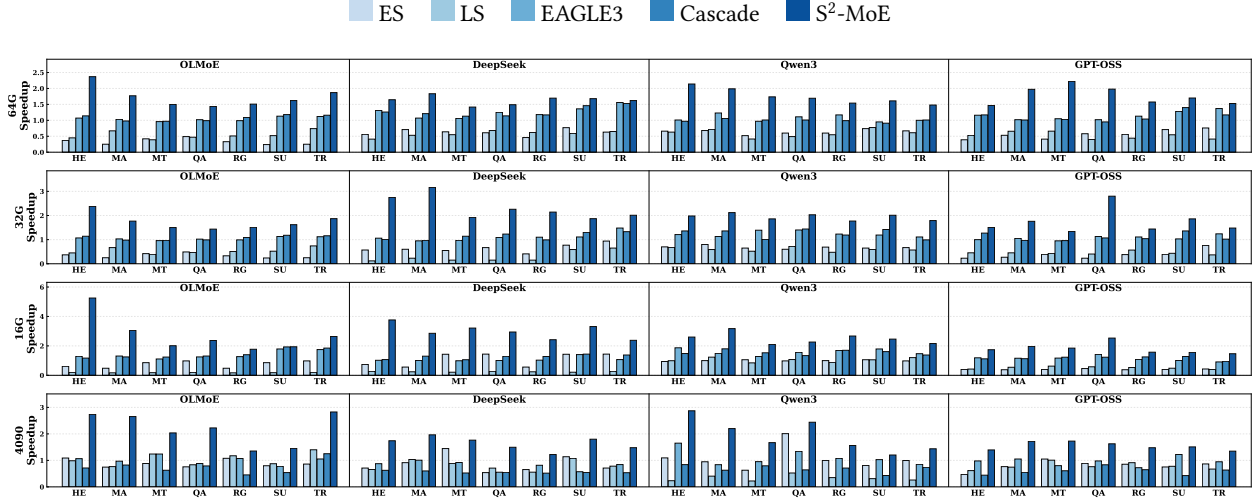


Figure 7: End-to-end speedup of different SD methods across MoE models, tasks, memory budgets on Jetson (16G/32G/64G), and RTX 4090.

and LongBench scores stay comparable to the original model. For the measured models, the PPL ratio remains close to 1.0 (1.012–1.013) and Top-1 agreement remains high (> 89%), with only small KL/logit shifts, indicating bounded output-level perturbation without observable degradation.

For GPT-OSS, we omit PPL/KL because standard next-token likelihood on raw or non-Harmony continuations is not a reliable quality signal: GPT-OSS is trained for the Harmony chat format and CoT/RL post-training objectives rather than raw language modeling [1]. Public reports document anomalously high raw-corpus PPL and poorly calibrated token log-probabilities for GPT-OSS under standard causal-LM evaluation [13, 19], and recent analysis treats this as a known artifact when Harmony-trained models are evaluated outside their intended format [45].

6.3 Ablation Study

6.3.1 Progressive Speedup Breakdown. Figure 8 visualizes the progressive speedup gains contributed by each component of S²-MoE. Across both Qwen3 and DeepSeek, each design choice consistently improves performance, and their combination yields the highest overall speedup, confirming that the three components are complementary rather than redundant.

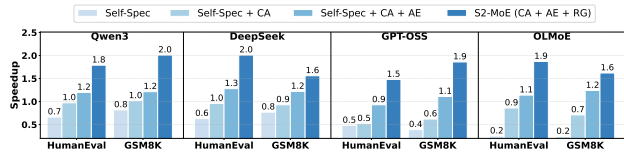


Figure 8: Ablation study on different models, showing the progressive contribution of each component in S²-MoE. CA: context-aligned self-speculative decoding; AE: adaptive expansion; RG: reuse-aware gating.

6.3.2 Effect of Individual Design Components.

Higher Draft Fidelity. We first examine the effect of context-aligned self-speculative decoding on draft fidelity. Table 3 compares acceptance rates with and without context alignment. Across both DeepSeek and OLMoE, enabling context alignment consistently improves acceptance rates on all evaluated tasks. The improvement is particularly pronounced on reasoning-heavy benchmarks such as GSM8K, where acceptance rates increase by up to 79%. These results indicate that sharing an identical KV cache between the draft and target effectively prevents error accumulation across speculative iterations, forming a critical foundation for efficient speculative decoding under lightweight drafts.

Table 3: Effect of context-aligned (CA) self-speculative decoding on acceptance rate (Acc.). Higher values indicate better draft fidelity.

Model	Dataset	w/o Context-Aligned	w/ Context-Aligned	Acceptance rate $\uparrow$
DeepSeek	HumanEval	43.01	52.91	+23.02%
	GSM8K	36.08	48.71	+35.01%
OLMoE	HumanEval	32.46	40.31	+24.18%
	GSM8K	33.01	59.09	+79.01%

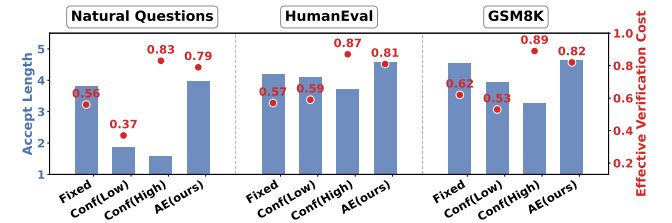


Figure 9: Comparison of different speculative expansion strategies across datasets. We report the acceptance length and the effective verification cost, defined as the fraction of expert activations contributing to accepted tokens.

Table 2: Quality comparison across MoE models: task accuracy, long-context quality, and distribution consistency.

Model	Method	Task accuracy					Long-context			Distribution consistency			
		GSM8K	HumanEval	ARC-E	ARC-C	Avg.	Gov.	NQA	Qasper	Mean KL ↓	RMS Δp ↓	Top-1 ↑	PPL ↓
OLMoE	Original	35.29	22.56	55.81	43.86	39.38	24.69	8.10	21.52	0	0	100	1.000
	S ² -MoE	34.49	24.39	55.71	43.86	39.61	24.14	7.90	23.30	0.048	5.20	89.89	1.012
DeepSeek	Original	66.64	48.98	63.06	56.03	58.68	30.30	11.86	27.40	0	0	100	1.000
	S ² -MoE	67.27	50.20	66.91	58.56	60.74	29.20	13.13	32.72	0.065	6.95	89.13	1.013

Model	Method	Task accuracy					Long-context			Distribution consistency			
		AIME24	AIME25	HMMT	GPQA	Avg.	Gov.	NQA	Qasper	Mean KL ↓	RMS Δp ↓	Top-1 ↑	PPL ↓
Qwen3	Original	68.89	54.44	36.67	79.04	59.76	30.38	15.37	28.05	0	0	100	1.000
	S ² -MoE	70.00	55.56	38.89	78.54	60.75	30.22	16.06	27.53	0.026	4.88	93.04	1.012
GPT-OSS	Original	61.11	62.22	55.83	72.02	62.80	23.42	13.34	31.36	–	–	–	–
	S ² -MoE	63.34	62.22	55.00	72.01	63.14	25.32	16.71	35.82	–	–	–	–

Less Redundant Verification. Figure 9 compares different speculative expansion strategies in terms of effective acceptance length and verification efficiency across multiple datasets. Fixed-width expansion achieves reasonable acceptance length but incurs substantial redundant verification. Confidence-based pruning exhibits a clear trade-off: a low threshold preserves acceptance but incurs heavy redundant verification, while a higher one reduces verification overhead but significantly limits acceptance length.

In contrast, our utility-guided adaptive expansion consistently achieves acceptance length comparable to or exceeding fixed-width expansion, while maintaining verification efficiency close to aggressive confidence pruning. Compared to confidence-based pruning, our method attains a strictly better acceptance–verification trade-off, effectively pushing the Pareto frontier by jointly considering acceptance likelihood and verification cost.

Higher Expert Reuse. Table 4 reports the effect of our reuse-aware expert gating on expert parameter reuse in DeepSeek. Across different datasets, enabling reuse-aware gating consistently reduces the reuse ratio, indicating substantially higher expert reuse during verification. This directly validates Observation 3 in Sec. 3 and confirms that reuse-aware gating effectively aligns expert selection across speculative tokens, thereby improving parameter locality.

6.3.3 Hyperparameter Sensitivity Analysis. We further study the sensitivity of two key hyperparameters in S²-MoE: the draft expert top- k and the reuse-aware gating cap.

Effect of draft expert top- k . The top row of Fig. 10 studies the effect of draft expert top- k , i.e., the number of experts activated per token in the self-speculative draft model, on Qwen3 and GPT-OSS on GSM8K. Increasing top- k improves acceptance length by providing higher-quality draft tokens, but also increases draft overhead, which can reduce overall speedup. Conversely, overly small top- k limits acceptance length and underutilizes speculative opportunities. These results indicate a clear trade-off, and motivate choosing a moderate top- k that balances draft cost and acceptance gain.

Table 4: Effect of reuse-aware expert gating on expert parameter reuse. Expert reuse is measured using the reuse ratio defined in Challenge 2 (Sec. 3).

Dataset	w/o RG	w/ RG	Reuse Improvement (%)
HumanEval	0.6017	0.4453	25.99
GSM8K	0.5893	0.3943	33.09

Effect of reuse-aware gating cap. The bottom row of Fig. 10 evaluates the impact of the reuse-aware gating cap on DeepSeek and OLMoE on HumanEval. When the cap is too small, only a limited number of experts are encouraged for reuse, while the remaining experts are still selected in a scattered manner, leading to insufficient improvement in expert reuse. On the other hand, an excessively large cap dilutes the bias by rewarding too many experts, effectively weakening the reuse signal and reducing its impact on verification efficiency. In practice, we find that setting the cap within a moderate range (e.g., 1.5 $\times$ to 2.5 $\times$ the top- k) provides a good balance between promoting expert reuse and preserving routing flexibility.

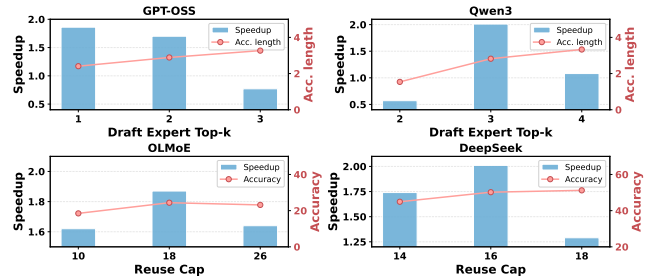


Figure 10: Sensitivity to key hyperparameters in S²-MoE.

7 Conclusion

In conclusion, we present S^2 -MoE, an efficient self-speculative decoding framework for Mixture-of-Experts models in memory-constrained edge environments. By alleviating the memory-bandwidth bottleneck through context alignment, utility-guided speculation, and reuse-aware gating, S^2 -MoE achieves **1.3× to 5.3×** speedup on Jetson Orin and **1.2× to 2.9×** on RTX 4090 over autoregressive decoding, and consistently outperforms state-of-the-art speculative baselines while maintaining comparable accuracy.

A End-to-End Raw Measurements

This appendix reports the raw throughput (Tok/s), acceptance length (Acc.), and speedup underlying Fig. 7. Tables 5–8 list these measurements on Jetson Orin NX 16 GB, Jetson AGX Orin 32 GB/64 GB, and RTX 4090.

Table 5: Raw measurements on NVIDIA Jetson Orin NX 16 GB.

Task	Method	DeepSeek			OLMoE			Qwen3			GPT-OSS		
		T/s	Acc	Spd	T/s	Acc	Spd	T/s	Acc	Spd	T/s	Acc	Spd
HE	Auto	0.55	1.00	1.00	2.30	1.00	1.00	0.47	1.00	1.00	0.95	1.00	1.00
	ES	0.41	4.73	0.74	1.40	3.08	0.61	0.45	5.00	0.94	0.38	2.77	0.40
	LS	0.15	1.70	0.27	0.46	1.06	0.20	0.47	5.08	0.99	0.41	1.58	0.42
	EAGLE3	0.56	2.43	1.02	2.96	1.23	1.29	0.89	1.15	1.87	1.14	1.81	1.19
	Cascade	0.58	1.10	1.07	2.69	1.26	1.17	0.70	1.96	1.48	1.07	1.00	1.12
MA	S ² -MoE	2.07	5.42	3.76	12.08	7.00	5.25	1.22	6.70	2.60	1.65	2.95	1.74
	Auto	0.69	1.00	1.00	2.22	1.00	1.00	0.41	1.00	1.00	0.99	1.00	1.00
	ES	0.39	4.36	0.56	1.06	3.15	0.48	0.41	4.71	1.00	0.38	1.63	0.38
	LS	0.17	1.56	0.24	0.38	1.05	0.17	0.51	3.38	1.23	0.54	2.42	0.55
	EAGLE3	0.70	1.54	1.01	2.91	1.29	1.31	0.61	1.21	1.49	1.15	1.21	1.16
MT	Cascade	0.90	1.13	1.30	2.77	1.33	1.25	0.74	1.66	1.80	1.12	1.37	1.13
	S ² -MoE	1.97	4.06	2.85	6.77	4.79	3.05	1.30	6.45	3.17	1.95	3.25	1.97
	Auto	0.49	1.00	1.00	2.10	1.00	1.00	0.48	1.00	1.00	1.00	1.00	1.00
	ES	0.70	3.14	1.43	1.80	2.52	0.86	0.51	3.64	1.06	0.40	2.98	0.40
	LS	0.10	1.22	0.21	0.38	4.08	0.18	0.41	2.58	0.85	0.63	2.24	0.62
QA	EAGLE3	0.48	1.75	0.99	2.33	1.19	1.11	0.62	1.15	1.28	1.17	1.81	1.17
	Cascade	0.52	1.77	1.06	2.60	1.30	1.24	0.74	1.89	1.53	1.23	1.18	1.23
	S ² -MoE	1.57	2.79	3.21	4.22	3.35	2.01	1.01	4.64	2.10	1.85	3.05	1.85
	Auto	0.61	1.00	1.00	2.09	1.00	1.00	0.50	1.00	1.00	0.97	1.00	1.00
	ES	0.88	2.67	1.44	2.05	1.89	0.98	0.49	3.80	0.98	0.45	1.56	0.46
RG	LS	0.16	1.88	0.27	0.42	4.72	0.20	0.53	3.64	1.07	0.56	1.50	0.58
	EAGLE3	0.61	1.77	1.01	2.61	1.10	1.25	0.78	1.81	1.56	1.38	1.31	1.42
	Cascade	0.79	1.58	1.29	2.74	1.20	1.31	0.67	1.70	1.34	1.20	1.24	1.23
	S ² -MoE	1.80	3.88	2.94	4.95	3.94	2.37	1.13	5.15	2.26	2.45	3.35	2.53
	Auto	0.58	1.00	1.00	2.12	1.00	1.00	0.49	1.00	1.00	0.98	1.00	1.00
SU	ES	0.33	2.67	0.56	1.02	1.89	0.48	0.49	3.80	1.00	0.37	1.56	0.38
	LS	0.14	1.88	0.24	0.36	2.00	0.17	0.43	2.78	0.88	0.52	1.28	0.53
	EAGLE3	0.60	1.91	1.03	2.69	1.09	1.27	0.82	1.07	1.69	1.05	1.09	1.08
	Cascade	0.75	1.63	1.28	2.96	1.16	1.40	0.83	1.81	1.70	1.22	1.13	1.25
	S ² -MoE	1.40	4.46	2.42	3.75	5.00	1.77	1.31	5.38	2.67	1.54	2.71	1.57
TR	Auto	0.47	1.00	1.00	2.11	1.00	1.00	0.48	1.00	1.00	0.98	1.00	1.00
	ES	0.68	4.36	1.43	1.81	3.15	0.86	0.51	4.71	1.06	0.39	1.63	0.40
	LS	0.10	1.56	0.21	0.38	1.05	0.18	0.51	3.06	1.05	0.47	1.06	0.48
	EAGLE3	0.67	1.74	1.41	3.77	1.09	1.79	0.86	1.82	1.79	0.99	1.81	1.01
	Cascade	0.68	1.53	1.44	4.07	1.07	1.93	0.77	1.82	1.61	1.26	1.83	1.28
MA	S ² -MoE	1.56	3.72	3.31	4.09	3.82	1.94	1.18	5.15	2.46	1.53	2.39	1.56
	Auto	0.61	1.00	1.00	2.14	1.00	1.00	0.48	1.00	1.00	0.98	1.00	1.00
	ES	0.88	3.14	1.44	2.10	2.52	0.98	0.47	3.64	0.98	0.42	2.98	0.43
	LS	0.17	1.22	0.27	0.43	1.70	0.20	0.58	3.87	1.20	0.39	1.51	0.40
	EAGLE3	0.66	1.46	1.07	3.75	1.09	1.75	0.70	1.06	1.47	0.90	1.75	0.91
MT	Cascade	0.85	1.52	1.38	3.96	1.11	1.85	0.66	1.82	1.38	0.94	1.64	0.95
	S ² -MoE	1.45	3.82	2.38	5.65	3.63	2.64	1.04	6.27	2.16	1.44	2.39	1.47

Table 6: Raw measurements on NVIDIA Jetson AGX Orin 32 GB.

Task	Method	DeepSeek			OLMoE			Qwen3			GPT-OSS		
		T/s	Acc	Spd	T/s	Acc	Spd	T/s	Acc	Spd	T/s	Acc	Spd
HE	Auto	2.30	1.00	1.00	31.27	1.00	1.00	0.78	1.00	1.00	1.12	1.00	1.00
	ES	1.31	4.62	0.57	11.57	1.03	0.37	0.55	4.14	0.70	0.26	2.54	0.23
	LS	0.28	6.92	0.12	14.07	6.55	0.45	0.53	4.46	0.68	0.51	1.96	0.45
	EAGLE3	2.44	2.46	1.06	33.46	1.27	1.07	0.95	1.50	1.21	1.12	2.36	1.00
	Cascade	2.32	1.96	1.01	35.65	1.31	1.14	1.07	1.02	1.36	1.43	2.43	1.27
MA	S ² -MoE	6.32	7.78	2.75	74.32	5.23	2.38	1.54	5.00	1.98	1.69	4.47	1.50
	Auto	2.38	1.00	1.00	31.34	1.00	1.00	0.79	1.00	1.00	1.13	1.00	1.00
	ES	1.43	3.91	0.60	7.83	1.07	0.25	0.63	3.19	0.80	0.31	1.88	0.27
	LS	0.55	6.37	0.23	21.00	4.63	0.67	0.47	3.25	0.59	0.51	1.87	0.45
	EAGLE3	2.27	1.87	0.95	32.28	1.35	1.03	0.89	1.95	1.13	1.19	1.89	1.05
MT	Cascade	2.31	1.79	0.97	30.71	1.26	0.98	1.07	1.94	1.36	1.10	1.16	0.97
	S ² -MoE	7.53	6.25	3.16	55.37	4.47	1.77	1.67	4.38	2.12	1.99	5.42	1.76
	Auto	2.24	1.00	1.00	31.28	1.00	1.00	0.78	1.00	1.00	1.15	1.00	1.00
	ES	1.23	3.16	0.55	13.14	2.24	0.42	0.51	3.69	0.65	0.45	1.07	0.39
	LS	0.34	1.18	0.15	12.20	4.08	0.39	0.41	2.35	0.52	0.49	2.47	0.43
QA	EAGLE3	2.18	1.91	0.97	30.03	1.15	0.96	1.08	1.02	1.39	1.09	1.22	0.95
	Cascade	2.56	1.76	1.14	30.34	1.28	0.97	0.79	1.83	1.01	1.10	1.13	0.96
	S ² -MoE	4.31	5.29	1.92	46.94	2.95	1.50	1.45	4.47	1.86	1.54	3.61	1.34
	Auto	2.19	1.00	1.00	31.30	1.00	1.00	0.76	1.00	1.00	1.10	1.00	1.00
	ES	1.47	2.09	0.67	15.34	1.92	0.49	0.45	3.49	0.60	0.25	2.21	0.23
RG	LS	0.33	1.54	0.15	14.71	4.72	0.47	0.55	3.47	0.72	0.44	1.24	0.40
	EAGLE3	2.42	1.96	1.10	30.96	1.17	0.99	0.96	1.99	1.23	1.24	1.18	1.11
	Cascade	2.18	1.61	0.99	34.09	1.15	1.09	0.93	1.90	1.19	1.17	1.14	1.04
	S ² -MoE	4.71	5.91	2.14	47.20	3.67	1.51	1.38	4.31	1.77	1.61	3.14	1.44
	Auto	2.25	1.00	1.00	31.37	1.00	1.00	0.79	1.00	1.00	1.14	1.00	1.00
SU	ES	1.74	3.91	0.77	7.53	1.07	0.24	0.51	3.19	0.65	0.45	1.88	0.39
	LS	1.33	6.37	0.59	16.31	4.63	0.52	0.47	3.33	0.59	0.49	1.83	0.43
	EAGLE3	2.50	1.61	1.11	35.45	1.07	1.13	0.94	1.83	1.19	1.18	1.23	1.03
	Cascade	2.93	1.71	1.30	37.02	1.12	1.18	1.12	1.67	1.42	1.55	1.24	1.36
	S ² -MoE	4.21	3.24	1.87	50.82	3.05	1.62	1.59	3.89	2.01	2.12	3.00	1.86
TR	Auto	2.26	1.00	1.00	31.32	1.00	1.00	0.78	1.00	1.00	1.10	1.00	1.00
	ES	2.12	3.16	0.94	7.83	2.24	0.25	0.52	3.69	0.67	0.84	1.07	0.76
	LS	1.47	1.18	0.65	23.18	4.08	0.74	0.44	3.60	0.57	0.41	1.16	0.37
	EAGLE3	3.34	1.55	1.48	35.08	1.04	1.12	0.86	1.30	1.11	1.37	1.81	1.24
	Cascade	3.00	1.46	1.33	36.34	1.22	1.16	0.77	1.90	0.99	1.12	1.73	1.02
MA	S ² -MoE	4.54	4.17	2.01	58.57	2.88	1.87	1.40	5.31	1.79	1.63	4.12	1.48

Table 7: Raw measurements on NVIDIA Jetson AGX Orin 64 GB.

		DeepSeek			OLMoE			Qwen3			GPT-OSS			
Task	Method	T/s	Acc	Spd	T/s	Acc	Spd	T/s	Acc	Spd	T/s	Acc	Spd	
HE	Auto	16.17	1.00	1.00	31.36	1.00	1.00	0.85	1.00	1.00	1.30	1.00	1.00	
	ES	9.06	8.00	0.56	11.60	1.03	0.37	0.56	6.67	0.66	0.51	2.33	0.39	
	LS	6.63	8.00	0.41	14.11	6.55	0.45	0.54	3.71	0.63	0.69	2.09	0.53	
	EAGLE3	21.18	2.69	1.31	33.55	1.27	1.07	0.86	1.01	1.01	1.50	1.01	1.16	
	Cascade	20.38	1.51	1.26	35.75	1.31	1.14	0.83	1.95	0.97	1.52	1.01	1.17	
MA	S ² -MoE	26.62	5.50	1.65	74.32	5.23	2.37	1.82	5.00	2.14	1.91	2.83	1.47	
	Auto	16.18	1.00	1.00	31.28	1.00	1.00	0.90	1.00	1.00	1.30	1.00	1.00	
	ES	11.48	2.80	0.71	7.82	1.07	0.25	0.61	3.00	0.68	0.69	1.67	0.53	
	LS	8.57	1.67	0.53	20.96	4.63	0.67	0.56	3.86	0.72	0.86	2.38	0.66	
	EAGLE3	17.31	1.32	1.07	32.22	1.35	1.20	0.93	1.11	1.17	1.23	1.33	1.65	1.07
MT	Cascade	19.57	1.40	1.21	30.66	1.26	1.08	0.96	1.07	1.06	1.32	1.38	1.81	1.01
	S ² -MoE	29.68	5.00	1.83	55.37	4.47	1.77	1.79	4.38	1.99	2.56	3.25	1.97	
	Auto	16.18	1.00	1.00	31.31	1.00	1.00	0.91	1.00	1.00	1.05	1.00	1.00	
	ES	10.36	1.29	0.64	13.15	2.24	0.42	0.47	1.70	0.52	0.43	2.00	0.41	
	LS	8.90	1.18	0.55	12.21	4.08	0.39	0.38	1.91	0.41	0.69	1.58	0.66	
QA	EAGLE3	17.15	1.67	1.06	30.06	1.15	0.96	0.92	1.13	0.97	1.10	1.11	1.05	
	Cascade	18.29	1.64	1.13	30.37	1.28	0.97	0.88	1.81	1.01	1.08	1.11	1.03	
	S ² -MoE	22.91	3.63	1.42	46.94	2.95	1.50	1.58	4.47	1.74	2.33	2.83	2.22	
	Auto	16.16	1.00	1.00	31.29	1.00	1.00	0.89	1.00	1.00	1.33	1.00	1.00	
	ES	9.86	2.60	0.61	15.33	1.92	0.49	0.54	3.53	0.60	0.77	1.25	0.58	
RG	LS	10.92	2.60	0.51	14.72	0.42	0.44	2.46	0.52	0.52	1.25	0.58	0.58	
	EAGLE3	20.29	2.63	1.73	25.31	1.25	1.91	1.02	1.09	1.82	1.11	1.35	1.29	1.02
	Cascade	18.42	1.43	1.44	30.98	1.21	0.99	0.90	1.67	1.01	1.26	1.13	0.95	
	S ² -MoE	24.08	3.61	1.49	49.42	3.50	1.41	1.51	4.42	1.69	2.63	3.05	1.98	
	Auto	16.16	1.00	1.00	31.30	1.00	1.00	0.91	1.00	1.00	1.31	1.00	1.00	
SU	ES	7.44	1.57	0.46	10.33	1.92	0.33	0.54	4.00	0.60	0.74	1.60	0.56	
	LS	10.02	3.00	0.62	15.96	2.00	0.50	0.50	2.82	0.55	0.58	1.83	0.44	
	EAGLE3	19.07	1.79	1.18	30.98	1.17	0.99	0.56	1.90	1.17	1.48	1.22	1.13	
	Cascade	18.91	1.61	1.17	34.11	1.15	1.09	0.90	1.93	0.99	1.37	1.13	1.04	
	S ² -MoE	27.40	4.38	1.70	47.20	3.67	1.51	1.40	4.31	1.54	2.06	2.71	1.57	
TR	Auto	16.17	1.00	1.00	31.28	1.00	1.00	0.90	1.00	1.00	1.30	1.00	1.00	
	ES	12.45	2.00	0.77	7.51	1.43	0.24	0.67	3.00	0.74	0.92	2.40	0.71	
	LS	9.54	1.29	0.59	16.27	3.40	0.52	0.40	0.70	0.77	0.71	1.64	0.55	
	EAGLE3	21.99	1.79	1.36	35.34	1.07	1.13	0.86	1.83	0.95	1.66	1.65	1.28	
	Cascade	23.61	1.75	1.46	36.91	1.12	1.18	0.82	1.70	0.91	1.82	1.67	1.40	
TR	S ² -MoE	27.17	1.35	1.68	50.67	3.05	1.62	1.45	3.89	1.61	2.21	2.75	1.70	
	Auto	16.18	1.00	1.00	31.30	1.00	1.00	0.89	1.00	1.00	1.27	1.00	1.00	
	ES	10.19	5.33	0.63	7.82	1.89	0.25	0.59	3.50	0.67	0.97	1.67	0.76	
	LS	10.12	1.67	0.22	10.54	0.54	0.36	0.41	0.51	0.51	0.51	1.11	0.11	
	EAGLE3	25.24	2.14	1.56	35.05	1.04	1.12	0.89	1.09	1.00	1.75	1.51	1.37	
TR	Cascade	24.76	1.52	1.53	36.30	1.22	1.16	0.90	1.68	1.01	1.49	1.51	1.17	
	S ² -MoE	26.21	3.94	1.62	58.53	2.88	1.87	1.32	5.31	1.48	1.94	2.83	1.53	

References

- [1] Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K Arora, Yu Bai, Bowen Baker, Haiming Bao, et al. 2025. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925* (2025).
- [2] Yuwei An, Zhuoming Chen, and Beidi Chen. [n. d.]. IFMoE: An Inference Framework Design for Fine-grained MoE. ([n. d.]).
- [3] AngelSlim. 2025. Qwen3-A3B-EAGLE3. https://huggingface.co/AngelSlim/Qwen3-a3B_eagle3. Hugging Face model checkpoint.
- [4] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Bangkok, Thailand, 3119–3137. doi:10.18653/v1/2024.acl-long.172
- [5] Jehyeon Bang, Eunyong Cho, Ranggi Hwang, Jinha Chung, and Minsoo Rhu. 2026. SpecMoE: A Fast and Efficient Mixture-of-Experts Inference via Self-Assisted Speculative Decoding. *arXiv preprint arXiv:2604.10152* (2026). Extended version of a paper accepted at DAC 2026.
- [6] Liangkun Chen, Zijian Wen, Tian Wu, Xiaoxi Zhang, and Chuan Wu. 2025. SP-MoE: Speculative Decoding and Prefetching for Accelerating MoE-based Model Inference. *arXiv preprint arXiv:2510.10302* (2025).
- [7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [8] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge. arXiv:1803.05457 [cs.AI] <https://arxiv.org/abs/1803.05457>
- [9] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. 2021. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168> 9 (2021).
- [10] Nobel Dhar, Robin Deng, Dan Lo, Xiaofeng Wu, Liang Zhao, and Kun Suo. 2024. An empirical analysis and resource footprint study of deploying large language models on edge devices. In *Proceedings of the 2024 ACM southeast conference*. 69–76.
- [11] Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, et al. 2024. Layerskip: Enabling early exit inference and self-speculative decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 12622–12642.
- [12] Zizhuo Fu, Xiaotian Guo, Wenxuan Zeng, Shuzhang Zhong, Yadong Zhang, Peiyu Chen, Runsheng Wang, Le Ye, and Meng Li. 2025. H 2 EAL: Hybrid-Bonding Architecture with Hybrid Sparse Attention for Efficient Long-Context LLM Inference. In *2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 1–9.
- [13] ggml-org contributors. 2025. Discussion of gpt-oss Likelihood / Perplexity Evaluation Artifacts. <https://github.com/ggml-org/llama.cpp/issues/15155>. GitHub issue.
- [14] ggml-org contributors. 2025. EAGLE-3 Speculative Decoding in llama.cpp. <https://github.com/ggml-org/llama.cpp/pull/18039>. GitHub pull request.
- [15] Zhenyu He, Zexuan Zhong, Tianle Cai, Jason Lee, and Di He. 2024. Rest: Retrieval-based speculative decoding. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 1582–1595.
- [16] HMMT Organization. 2025. Harvard-MIT Mathematics Tournament (HMMT), February 2025. <https://www.hmmt.org>. Accessed: 2025.
- [17] Haochen Huang, Shuzhang Zhong, Zhe Zhang, Shuangchen Li, Dimin Niu, Hongzhong Zheng, Runsheng Wang, and Meng Li. 2025. HD-MoE: Hybrid and Dynamic Parallelism for Mixture-of-Expert LLMs with 3D Near-Memory Processing. In *2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 1–9.
- [18] Zongle Huang, Lei Zhu, Zongyuan Zhan, Ting Hu, Weikai Mao, Xianzhi Yu, Yongpan Liu, and Tianyu Zhang. 2025. MoESD: Unveil Speculative Decoding’s Potential for Accelerating Sparse MoE. *arXiv preprint arXiv:2505.19645* (2025).
- [19] Hugging Face Transformers contributors. 2025. Anomalously High Perplexity / Poorly Calibrated Log-Probabilities for gpt-oss under Standard Causal-LM Evaluation. <https://github.com/huggingface/transformers/issues/40990>. GitHub issue.
- [20] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*. PMLR, 19274–19286.
- [21] Shuhuai Li, Jianghao Lin, Dongdong Ge, and Yinyu Ye. 2026. MoE-SpAc: Efficient MoE Inference Based on Speculative Activation Utility in Heterogeneous Edge Scenarios. *arXiv preprint arXiv:2603.09983* (2026).
- [22] Xiangyu Li, Yuanchun Li, Yuanzhe Li, Ting Cao, and Yunxin Liu. 2024. Flexnn: Efficient and adaptive dnn inference on memory-constrained edge devices. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*. 709–723.
- [23] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2025. Eagle-3: Scaling up inference acceleration of large language models via training-time test. *arXiv preprint arXiv:2503.01840* (2025).
- [24] Aixun Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Daya Guo, et al. 2024. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434* (2024).
- [25] Aixun Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).
- [26] Fangcheng Liu, Yehui Tang, Zhenhua Liu, Yunsheng Ni, Duyu Tang, Kai Han, and Yunhe Wang. 2024. Kangaroo: Lossless self-speculative decoding for accelerating llms via double early exiting. *Advances in Neural Information Processing Systems* 37 (2024), 11946–11965.
- [27] Xiaoxuan Liu, Lanxiang Hu, Peter Bailis, Alvin Cheung, Zhijie Deng, Ion Stoica, and Hao Zhang. 2023. Online speculative decoding. *arXiv preprint arXiv:2310.07177* (2023).
- [28] LMSYS Organization. 2025. EAGLE3 GPT-OSS-120B. <https://huggingface.co/lmsys/EAGLE3-gpt-oss-120b-bf16>. Hugging Face model checkpoint.
- [29] Mathematical Association of America. 2024. American Invitational Mathematics Examination (AIME) 2024. <https://www.maa.org/math-competitions/aime>. Accessed: 2024.
- [30] Mathematical Association of America. 2025. American Invitational Mathematics Examination (AIME) 2025. <https://www.maa.org/math-competitions/aime>. Accessed: 2025.
- [31] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. Pointer Sentinel Mixture Models. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=Byj72udxe>
- [32] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, et al. 2024. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 932–949.
- [33] Niklas Muennighoff, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Jacob Morrison, Seonwon Min, Weijia Shi, Pete Walsh, Oyvind Tafjord, Nathan Lambert, et al. 2024. Olmo: Open mixture-of-experts language models. *arXiv preprint arXiv:2409.02060* (2024).
- [34] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. 2023. GPQA: A Graduate-Level Google-Proof Q&A Benchmark. arXiv:2311.12022 [cs.AI] <https://arxiv.org/abs/2311.12022>
- [35] Anish Saxena, Po-An Tsai, Hritvik Taneja, Aamer Jaleel, and Moinuddin Qureshi. 2025. Utility-Driven Speculative Decoding for Mixture-of-Experts. *arXiv preprint arXiv:2506.20675* (2025).
- [36] sgl-project. 2025. SpecForge: Efficient Speculative Decoding Framework. <https://github.com/sgl-project/SpecForge>. GitHub repository.
- [37] Wenfeng Wang, Jiacheng Liu, Xiaofeng Hou, Xinfeng Xia, Peng Tang, Mingxuan Zhang, Chao Li, and Minyi Guo. 2025. MoE-SpeQ: Speculative Quantized Decoding with Proactive Expert Prefetching and Offloading for Mixture-of-Experts. *arXiv preprint arXiv:2511.14102* (2025).
- [38] Zhibin Wang, Zhonghui Zhang, Yuhang Zhou, Zibo Wang, Mo Zhou, Peng Jiang, Weilin Cai, Chengying Huan, Rong Gu, Sheng Zhong, et al. 2025. Accelerating Mixture-of-Experts Inference by Hiding Offloading Latency with Speculative Decoding. *arXiv preprint arXiv:2508.21706* (2025).
- [39] wantsleep. 2025. OLMoE_1B_7B_Eagle3. https://huggingface.co/wantsleep/OLMoE_1B_7B_Eagle3. Hugging Face model checkpoint.
- [40] Heming Xia, Zhe Yang, Qingxiu Dong, Peiyi Wang, Yongqi Li, Tao Ge, Tianyu Liu, Wenjie Li, and Zhifang Sui. 2024. Unlocking Efficiency in Large Language Model Inference: A Comprehensive Survey of Speculative Decoding. arXiv:2401.07851 [cs.CL] <https://arxiv.org/abs/2401.07851>
- [41] Daliang Xu, Wangsong Yin, Hao Zhang, Xin Jin, Ying Zhang, Shiyun Wei, Mengwei Xu, and Xuanzhe Liu. 2024. Edgellm: Fast on-device llm inference with speculative decoding. *IEEE Transactions on Mobile Computing* (2024).
- [42] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).
- [43] A. Yang, B. Yang, B. Hui, B. Zheng, B. Yu, C. Zhou, C. Li, C. Li, D. Liu, F. Huang, et al. 2024. Qwen2 Technical Report. *arXiv preprint arXiv:2407.10671* (2024). <https://arxiv.org/abs/2407.10671> Version: 2024-07.
- [44] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. 2024. Qwen2. 5 technical report. *arXiv preprint arXiv:2412.15115* (2024).

- [45] Artur Zagitov, Alexander Miasnikov, Maxim Krutikov, Vladimir Aletov, Gleb Molodtsov, Nail Bashirov, Artem Tsedenov, and Aleksandr Beznosikov. 2026. Rethinking the Role of Tensor Decompositions in Post-Training LLM Compression. *arXiv preprint arXiv:2606.03465* (2026).
- [46] Linxiao Zeng, Haoyun Deng, Kangyuan Shu, and Shizhen Wang. 2025. Self-Speculative Biased Decoding for Faster Live Translation. *arXiv preprint arXiv:2509.21740* (2025).
- [47] Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, and Sharad Mehrotra. 2023. Draft & Verify: Lossless Large Language Model Acceleration via Self-Speculative Decoding. *CoRR* abs/2309.08168 (2023). <https://doi.org/10.48550/arXiv.2309.08168>
- [48] Juntao Zhao, Wenhao Lu, Sheng Wang, Lingpeng Kong, and Chuan Wu. 2025. QSpec: Speculative Decoding with Complementary Quantization Schemes. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (Eds.). Association for Computational Linguistics, Suzhou, China, 4779–4795. doi:10.18653/v1/2025.emnlp-main.240
- [49] Peirong Zheng, Wenchao Xu, and Haozhao Wang. 2026. Self-Speculative Decoding for On-device MoE Acceleration. In *Proceedings of the ACM Web Conference 2026*. 5155–5164. doi:10.1145/3774904.3792218
- [50] Shuzhang Zhong, Zebin Yang, Ruihao Gong, Runsheng Wang, Ru Huang, and Meng Li. 2024. Propd: Dynamic token tree pruning and generation for llm parallel decoding. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. 1–8.
- [51] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M Dai, Quoc V Le, James Laudon, et al. 2022. Mixture-of-experts with expert choice routing. *Advances in Neural Information Processing Systems* 35 (2022), 7103–7114.
- [52] Xiangwen Zhuge, Xu Shen, Zeyu Wang, Fan Dang, Xuan Ding, Danyang Li, Yahui Han, Tianxiang Hao, and Zheng Yang. 2025. SpecOffload: Unlocking Latent GPU Capacity for LLM Inference on Resource-Constrained Devices. *arXiv preprint arXiv:2505.10259* (2025).